

Lab 4: Connecting to the Cloud

Creating an end-to-end IoT system from device to server to application.

Developed by Andrea David, Harikrishna Kuttivelil, and Firouz Vafadari.

In this lab, you will leverage your previous work and collaborate with your team to create a working end-to-end IoT system that collects environmental data, stores it in a database, and displays it on a web server. This lab will require the use of your personal computer, however, you may also borrow one from the [UCSC library](#). Each team member will need to submit their own write-up and indicate their role and tasks in the project.

Before you begin, please read the *Lab Write-Up* section, as you are encouraged to document your process as you carry out this lab assignment. This assignment is worth **20%** of your final grade.

Check-off due: June 6, 2025

Write-up due: June 12, 2025

Lab Procedure

Part 1: Setting Up a Web Server & Getting Started

Helpful Resources

- [XAMPP](#)
- [XAMPP tutorial: installation and first steps](#)
- [phpMyAdmin Tutorial \(YouTube playlist\)](#)
- [phpMyAdmin Documentation](#)
- [What Is a Database?](#)
- [Python MySQL Tutorial](#)

Procedure

1. After consulting the above resources, familiarize yourself with XAMPP and its components, including the Apache web server and the MySQL database server.
2. Download the appropriate version of XAMPP for your operating system on your WiFi-enabled computer.
3. After starting the Apache and MySQL services using the XAMPP Control Panel, verify that XAMPP is working by navigating to <http://localhost/> in your web browser.
4. We will use phpMyAdmin to manage our MySQL database because of its user-friendly interface. Explore phpMyAdmin by navigating to <http://localhost/phpmyadmin/> and

logging in using the default username and password. Experiment with creating a new database and tables and trying out SQL commands.

Outcomes

- You should have a database that is hosted on a separate machine from your Raspberry Pi devices.
- You should be able to manage and explore your database via XAMPP on <http://localhost/>.
- You should be able to create new databases and tables and manipulate them via SQL commands.

Part 2: Creating and Connecting to a Database

Helpful Resources

- [Getting started with Python connector](#)

Procedure

1. To collect sensor measurements and store them in a database, we will use the two configurations from the polling and token-ring-based approaches you created in Lab 3.
2. Using phpMyAdmin, create a new database called **piSenseDB** with three tables named **sensor_readings1**, **sensor_readings2**, and **sensor_readings3** that will store the *temperature*, *humidity*, *wind speed*, and *soil moisture* measurements, along with a *timestamp* for each set of measurements, from the three Raspberry Pis.
3. Recreate your polling and token-ring-based configurations of Raspberry Pis and modify your **primary.py** and **token-ring.py** scripts from Lab 3 to connect to **piSenseDB** using the Python MySQL connector module. In the polling-based approach, the primary Pi should be designated as the database connector, while in the token-ring-based approach, any Pi should be able to be designated as the database connector.
4. Consider what network you will use to connect your sensing devices and your database-hosting machine.
HINT: What kind of network infrastructure can you use to both connect between your devices and database, while also being able to connect to the wider Internet as necessary for Part 3 later in this project?
5. Implement robustness in your systems. See the Note on Robustness for more information.
6. The Pi designated as the connector should insert the sensor data and timestamp from all three Pis into the appropriate tables in the database by executing an INSERT query. For example, in the polling-based approach, the Primary Pi should insert its own sensor data and timestamp into the table named **sensor_readings1** and insert the sensor data and timestamps from the two Secondary Pis into **sensor_readings2** and **sensor_readings3**.
7. Continue collecting sensor data and storing it in the database tables after each round.

Note on Robustness

In designing robust systems, the primary functions of the system should be maintained as much as possible in the face of disruptive events (like disconnections) and unideal system configurations. The primary function of the systems you design in this lab are to collect data and send it to the database. This primary function should be maintained even if 1-2 devices out of the 3 devices in your system disconnect. Disruptive cases include, but are not limited to:

- *Both configurations:* Devices starting up in random order.
- *Polling-based configuration:* One or both of the secondary devices failing or disconnecting.
- *Token ring-based configuration:* One or both of the devices not connected to the database failing.
- *Token ring-based configuration:* The device designated as the database connector failing or disconnecting.

How you handle these and other failure cases are up to you, but you must justify and demonstrate how you maintain system integrity and functionality despite failures.

Outcomes

- You should have selected a network infrastructure to connect your devices in order to communicate between your Raspberry Pis, between your Pis and your database machine, and, for Part 3, between your database machine and the Internet.
- You should have adapted your **sensor_polling.py** and **token_ring.py** code to connect to the database to store sensor information (<timestamp, temperature, humidity, soil moisture, wind>) as specified in the procedure.
- You should have implemented sufficient robustness as specified in the *Note on Robustness*.

Part 3: Displaying Sensor Measurements on a Web Application

Helpful Resources

- [Flask](#)
- [Flask Tutorial \(YouTube playlist\)](#)
- [HTML Intro](#)
- [Matplotlib](#)
- [Binning Data in Python](#)

Procedure

1. Consult the above resources to familiarize yourself with the Flask framework.
2. Re-familiarize yourself with the process of pulling weather data using the OpenMeteo or NWS APIs from Lab Project #1.
3. Create a script called **web-app.py** that retrieves the Raspberry Pi sensor measurements from the three tables in **piSenseDB** and retrieves the day's forecast from an online weather service. For each measurement type, it should display the sensor data per device and the average across devices (represented as time-series data), along with the day's forecasted

weather (represented as a single value, as it is likely constant for the duration of your system's duration), in a graph on a website using Flask.

- a. Given your timestamps for the data, you should organize your graph with timestamps available. You may need to consider binning your data (see *Helpful Resources*).
 - b. You should always present the average across devices, not including the forecasted values, for each of the measurements.
 - c. If you are unable to find forecast counterparts for specific measurements, you do not need to include them (e.g., soil moisture measurements may not be available so you will not be able to include a forecast for them).
 - d. Consider how you want to showcase your time-series sensor data alongside the forecasted data in each graph in a meaningful way.
4. Your website should contain a meaningfully descriptive title and show the collected temperature, humidity, wind speed, and soil moisture measurements alongside available forecasts in four separate time series graphs.
 5. Configure Flask to make your website externally visible so it can be accessed from a web browser running on another computer.

Outcomes

- You should have a script **web-app.py** in which you display sensor data per device and the average.
- You should be able to display weather service-provided forecast data alongside the sensor data.
- Your web application should have a descriptive title and your interface should be neat and legible and intentionally designed.
- Your web application should be externally accessible.

Extra Credit: Creating an Interesting & Meaningful Web App

By achieving the outcomes listed at the end of Parts 1-3 of this project, you will have satisfied the requirements. However, you may also earn extra credit by adding additional functionality and razzle-dazzle (i.e., pizzazz) to your web application. Think of this endeavor as turning your project into a more useful and interactive tool or product by implementing meaningful extra features or components. You may want to consider a specific use case or application domain and develop your application and system towards that purpose. This could include more insights with the data, integration with other sources or systems, or more interactive front-ends.

Your efforts will be evaluated by the utility, significance, and/or intricacy (i.e., pizzazz) of the extra features or components that you implement atop the requirements. You can receive up to 10% extra credit on both the write-up and check-off.

Lab Write-Up

Write-ups for your labs should provide comprehensive documentation of your work. They should be written as a tutorial or technical blog post with the goal of serving as a guide or document for someone unfamiliar with the lab assignment. Include screenshots, diagrams, or pictures where necessary to achieve this goal. You will be evaluated on the quality of your write-up and how well you meet the writing objectives described below.

Writing Objectives

As you write your paper, consider the following outline:

1. (30 points) Overall Design
 - a. Describe each component and its functions in the IoT system.
 - b. Include a diagram of the whole system, including the database on your machine, showing how they interact.
 - c. Include a picture of your whole system, labeling the devices and components as they are described in the diagram.
2. (20 points) Web Server
 - a. Describe how you configured your web server. Were you able to run it out of the box, or did you need to configure additional settings?
3. (30 points) Database Implementation
 - a. Describe how you designed your database. What tables did you create? Use a diagram to show their entries.
 - b. What data representation did you use?
 - c. What operations did you use to create and consult your database?
4. (40 points) Adapting the Polling vs. Token-Ring Configurations
 - a. Compare the polling and token-ring-based approaches in a scenario where not all the nodes have data to send and in a scenario where all nodes always have data to send.
 - b. How did you implement better robustness into your designs? How did your approaches differ between configurations?
 - c. What failure cases do your robustness features address and how do they address them?
 - d. How do the two approaches compare in terms of fairness when it comes to allocating access to the shared medium?
 - e. What other topologies and configurations might you consider for this project and what are their strengths and weaknesses?
5. (20 points) Web Application
 - a. How did you display constant forecast data alongside time-varying sensor data (including the average) in each graph? What was your reasoning?
 - b. What limitations did you encounter with your design approach and how did you get around them? What other tools not mentioned in this manual did you use to accomplish your web application?

6. (15 points) Extra Credit
 - a. What extra features or components did you implement and what purpose do they serve? Who or what kind of user community would appreciate them?
 - b. Describe how your extra features or components work. What were the design choices you made and how did you implement them? (Be sure to include any pictures, screenshots, or diagrams that would help us understand.)
7. (10 points) Conclusion
 - a. Summarize the takeaways you learned in this lab.

Check-off & Submission

To receive full credit for this lab project, you must do the following:

- **Check-off (in lab, before deadline).** Demonstrate Parts 2 and 3 of this lab project to your TA and answer a few questions about your process and work during a lab section. You will be required to demonstrate your system's response to unordered device startups and device disconnections.
- **Code submission (via Canvas, after check-off).** Include your modified *primary.py* and *token-ring.py* scripts, your *web-app.py* script, and any other supporting scripts you used in a file named *CSE157-Lab4.zip* and upload it to the assignment portal on Canvas right after you get checked off.
- **Write-up submission (via Canvas, before deadline).** Upload a PDF of your lab write-up by 11:59 pm by the deadline to the same assignment portal.

Check off 10% of your course grade

Write-up + Code 10% of your course grade